

TypeScript - Principiante

Tema 10: Modificadores de acceso

20 ejercicios progresivos, sin soluciones, para practicar public, private, protected, readonly, encapsulacion y control de acceso en clases TypeScript.

Ejercicio 1 - Propiedad publica

Crea una clase Person con una propiedad publica name.

Requisitos

- Inicializa name en el constructor.
- Lee y modifica name desde fuera de la clase.

Ejemplo de ejecucion

Nombre: Ana

Nuevo nombre: Laura

Objetivo

Comprender el acceso public.

Pista

Los miembros public pueden utilizarse desde fuera de la clase.

Ejercicio 2 - Saldo privado

Crea una clase BankAccount con una propiedad privada balance.

Requisitos

- No accedas directamente a balance desde fuera.
- Agrega un metodo para consultar el saldo.

Ejemplo de ejecucion

Saldo: \$2500

Objetivo

Practicar private.

Pista

Los miembros private solo son accesibles dentro de la clase.

Ejercicio 3 - Depositar

Amplia BankAccount con un metodo publico deposit que modifique el saldo privado.

Requisitos

- Rechaza cantidades menores o iguales a cero.
- El saldo sigue siendo private.

Ejemplo de ejecucion

Deposito: \$500 | Saldo: \$3000

Objetivo

Practicar encapsulacion.

Pista

Expone operaciones, no necesariamente los datos internos.

Ejercicio 4 - Retirar

Agrega withdraw a BankAccount.

Requisitos

- No permite retirar mas que el saldo.
- No permite cantidades invalidas.
- balance permanece private.

Ejemplo de ejecucion

Retiro realizado | Saldo: \$1800

Objetivo

Aplicar reglas mediante metodos publicos.

Pista

La clase controla como cambia su estado interno.

Ejercicio 5 - Password privado

Crea User con name publico y password privado.

Requisitos

- Agrega un metodo para verificar una contrasena.
- No expongas el password directamente.

Ejemplo de ejecucion

Password correcto: true

Objetivo

Proteger informacion interna.

Pista

En lugar de devolver la contrasena, ofrece una operacion de verificacion.

Ejercicio 6 - Readonly id

Crea Product con id readonly, name y price.

Requisitos

- id se establece al crear el producto.
- name y price pueden cambiar.
- Incluye como comentario un intento invalido de cambiar id.

Ejemplo de ejecucion

Producto 10 - Teclado

Objetivo

Practicar readonly.

Pista

readonly permite lectura pero impide reasignacion.

Ejercicio 7 - Public explicito

Crea Employee declarando explícitamente miembros public.

Requisitos

- Incluye id, name y department.
- Accede a ellos desde fuera.

Ejemplo de ejecución

100 - Carlos - Desarrollo

Objetivo

Reconocer public explicito e implícito.

Pista

En TypeScript public es el modificador predeterminado.

Ejercicio 8 - Stock privado

Crea Product con stock privado.

Requisitos

- Agrega addStock, removeStock y getStock.
- Nunca permitas stock negativo.

Ejemplo de ejecución

Stock actual: 12

Objetivo

Encapsular una propiedad con reglas.

Pista

Las modificaciones deben pasar por los métodos de la clase.

Ejercicio 9 - Protected basico

Crea una clase Person con una propiedad protected name y una clase Employee que herede de Person.

Requisitos

- Employee debe poder usar name.
- Intenta acceder desde fuera como comentario.

Ejemplo de ejecucion

Empleado: Ana

Objetivo

Practicar protected.

Pista

protected permite acceso en la clase y sus clases derivadas.

Ejercicio 10 - Salario protegido

Crea Employee con salary protected y Manager que herede de Employee.

Requisitos

- Manager puede calcular un bono usando salary.
- salary no debe ser accesible directamente desde fuera.

Ejemplo de ejecucion

Bono: \$5000

Objetivo

Usar protected en herencia.

Pista

protected es util cuando las subclases necesitan el dato.

Ejercicio 11 - Private vs protected

Crea una clase base con una propiedad private y otra protected. Crea una clase hija e intenta usar ambas.

Requisitos

- Deja como comentario el acceso invalido.
- Demuestra cual propiedad si puede utilizar la clase hija.

Ejemplo de ejecucion

Propiedad protected accesible desde la subclase

Objetivo

Distinguir private de protected.

Pista

private no esta disponible directamente en una subclase.

Ejercicio 12 - Constructor abreviado

Crea una clase Customer usando parameter properties en el constructor.

Requisitos

- Declara id readonly.
- name publico.
- email privado.
- Agrega un metodo para obtener el email.

Ejemplo de ejecucion

Cliente 1 - Laura

Objetivo

Practicar modificadores directamente en parametros del constructor.

Pista

TypeScript puede declarar e inicializar propiedades desde el constructor.

Ejercicio 13 - Configuración inmutable

Crea AppConfig con propiedades readonly apiUrl y environment.

Requisitos

- Ambas se reciben en el constructor.
- No pueden modificarse después.

Ejemplo de ejecución

Entorno: production

Objetivo

Usar readonly para configuración estable.

Pista

readonly es apropiado cuando un valor se fija al crear la instancia.

Ejercicio 14 - Contador encapsulado

Crea Counter con count privado.

Requisitos

- increment y decrement son públicos.
- getCount permite consultar.
- No permitas valores menores a cero.

Ejemplo de ejecución

Contador: 4

Objetivo

Practicar estado interno controlado.

Pista

El consumidor no necesita modificar count directamente.

Ejercicio 15 - Carrito protegido

Crea ShoppingCart con items protected y una subclase DiscountCart.

Requisitos

- ShoppingCart permite agregar productos.
- DiscountCart usa items para calcular un total con descuento.
- No accedas a items desde fuera.

Ejemplo de ejecucion

Total con descuento: \$1800

Objetivo

Aplicar protected a colecciones internas.

Pista

Una subclase puede reutilizar el estado protegido.

Ejercicio 16 - Cuenta con PIN

Crea Account con balance y pin privados.

Requisitos

- withdraw requiere el PIN correcto.
- No expongas balance ni pin directamente.
- Crea getBalance que tambien requiera PIN.

Ejemplo de ejecucion

Acceso autorizado | Saldo: \$3200

Objetivo

Combinar encapsulacion con validacion.

Pista

Los metodos publicos deciden cuando se permite acceder al estado.

Ejercicio 17 - Entidad base

Crea una clase base Entity con id readonly y createdAt protected.

Requisitos

- Crea User y Product heredando de Entity.
- Las clases hijas pueden usar createdAt.
- id no puede cambiar.

Ejemplo de ejecucion

Usuario 10 - Creado: 2026-09-28

Objetivo

Combinar readonly, protected y herencia.

Pista

Selecciona el modificador segun quien necesita acceder al dato.

Ejercicio 18 - Repositorio encapsulado

Crea UserRepository con una coleccion privada de usuarios.

Requisitos

- Expone getAll, getById, add y delete.
- No permite acceso directo al array interno.
- getAll no debe permitir modificar accidentalmente la coleccion interna.

Ejemplo de ejecucion

Usuarios registrados: 5

Objetivo

Encapsular colecciones internas.

Pista

Considera devolver una copia de la coleccion.

Ejercicio 19 - Servicio y repositorio

Crea UserRepository con datos privados y UserService que lo utilice.

Requisitos

- Repository controla la coleccion.
- Service expone operaciones de negocio.
- Los consumidores no deben acceder a los datos internos.
- Usa modificadores adecuados en ambas clases.

Ejemplo de ejecucion

Usuario creado correctamente

Objetivo

Practicar separacion de responsabilidades y acceso.

Pista

No todo debe ser public solo porque otra clase existe.

Ejercicio 20 - Sistema bancario

Construye Customer, BankAccount y BankService aplicando modificadores de acceso apropiados.

Requisitos

- Los ids deben ser readonly.
- El saldo de una cuenta debe estar encapsulado.
- Los datos sensibles no deben ser publicos.
- BankAccount permite depositar y retirar con validaciones.
- BankService administra cuentas mediante una coleccion no accesible directamente.
- Permite transferencias entre cuentas.
- Una transferencia no puede dejar saldo negativo.
- Usa public, private, protected o readonly justificadamente.
- No uses any.

Ejemplo de ejecucion

Transferencia: \$500

Origen: \$1500

Destino: \$3000

Objetivo

Integrar modificadores de acceso, clases, encapsulacion y reglas de negocio.

Pista

Antes de escribir codigo decide quien debe poder leer o modificar cada dato.